

Ravan Muradzada

- 1) The wheel group in Linux is technically just a normal group like any other group in the system. There is no huge built-in difference in how the kernel treats it. The difference comes from how the system administrator configures it. The wheel group is commonly configured in the `/etc/sudoers` file to control administrative access. Inside that file, there is usually a line like:

```
%wheel ALL=(ALL) ALL
```

This line means that any user who is a member of the wheel group is allowed to use `sudo` to run commands as root or as another user.

- 2) First, to let users use these commands, we need to find their paths using `which` command:

```
[root@rhel10 ~]# which ip && which netstat && which tcpdump
/usr/sbin/ip
/usr/bin/netstat
/usr/sbin/tcpdump
```

Let's create a new user named "networker":

```
[root@rhel10 ~]# useradd networker && passwd networker
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
[root@rhel10 ~]# tail -n1 /etc/passwd
networker:x:1003:1003:./home/networker:/bin/bash
```

We go to `/etc/sudoers.d` directory and create a file named `networker` here and open it to write these:

```
Cmnd_Alias NETWORK_COMMANDS = /usr/sbin/ip address show, /usr/sbin/ip route, /usr/bin/netstat, /usr/sbin/tcpdump
networker ALL=(ALL) NETWORK_COMMANDS
```

BTW: Here command name cannot contain lowercase letters. (It failed when it is)

Then we can run the commands specified here with the usage of `sudo`:

(I will not show full output, because it is long)

```
[networker@rhel10 ~]$ sudo ip address show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

```
[networker@rhel10 ~]$ sudo ip route
default via 192.168.88.2 dev ens160 proto dhcp src 192.168.88.132 metric 100
192.168.88.0/24 dev ens160 proto kernel scope link src 192.168.88.132 metric 100
```

```
[networker@rhel10 ~]$ sudo netstat
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 rhel10.1:ssh            192.168.88.1:54075      ESTABLISHED
tcp        0     36 rhel10.1:ssh            192.168.88.1:51334      ESTABLISHED
tcp        0      0 rhel10.1:ssh            192.168.88.1:54076      ESTABLISHED
tcp        0      0 rhel10.1:ssh            192.168.88.1:51335      ESTABLISHED
```

```
[networker@rhel10 ~]$ sudo tcpdump
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens160, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

BTW: Among these commands, only `tcpdump` need sudo permission to run. Others can be run without sudo.

3) I have created a directory inside root directory named `/WorkingDir` and created my script file inside it:

```
#!/bin/bash

echo "Host IP Addresses:"
ip -brief address

echo ""
echo "Open Ports:"
ss -tuln
```

This is the content of script file.

```
[root@rhel10 WorkingDir]# chmod u+x script.sh
[root@rhel10 WorkingDir]# ll
total 4
-rwxr--r--. 1 root root 94 Mar  8 17:50 script.sh
```

As you can see, only root can execute this command. Others need to have sudo access for it.

Let's create `scriptrunner` user:

```
[root@rhel10 ~]# useradd scriptrunner && passwd scriptrunner
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
```

Let's add him to sudoers.d. We create a file named `scriptrunner` inside `sudoers.d` directory and type it there:

```
scriptrunner ALL=(ALL) /WorkingDir/script.sh
```

When this user goes the specified directory and runs the script with `sudo`, it will work:

```
[scriptrunner@rhel10 WorkingDir]$ ./script.sh #Without 'sudo' it doesn't work
-bash: ./script.sh: Permission denied
[scriptrunner@rhel10 WorkingDir]$ sudo ./script.sh #With 'sudo' it works
[sudo] password for scriptrunner:
Host IP Addresses:
lo UNKNOWN 127.0.0.1/8 ::1/128
ens160 UP 192.168.88.132/24 fe80::20c:29ff:fe8b:9a95/64

Open Ports:

```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
udp	UNCONN	0	0	0.0.0.0:5353	0.0.0.0:*
udp	UNCONN	0	0	127.0.0.1:323	0.0.0.0:*
udp	UNCONN	0	0	:::5353	:::*
udp	UNCONN	0	0	:::323	:::*
tcp	LISTEN	0	128	0.0.0.0:22	0.0.0.0:*
tcp	LISTEN	0	4096	127.0.0.1:631	0.0.0.0:*
tcp	LISTEN	0	128	:::22	:::*
tcp	LISTEN	0	4096	:::631	:::*
tcp	LISTEN	0	4096	:::9090	:::*

```
[root@rhel10 ~]# useradd lisa && passwd lisa
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
[root@rhel10 ~]# useradd lara && passwd lara
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
[root@rhel10 ~]# useradd sara && passwd sara
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
[root@rhel10 ~]# tail -n3 /etc/passwd
lisa:x:1005:1005:./home/lisa:/bin/bash
lara:x:1006:1006:./home/lara:/bin/bash
sara:x:1007:1007:./home/sara:/bin/bash
```

4)

5) I have set passwords in Task 4. Looking at above, I did it there using `passwd` command.

6) `[root@rhel10 ~]# groupadd sales`

```
[root@rhel10 ~]# usermod -aG sales lisa && id lisa
uid=1005(lisa) gid=1005(lisa) groups=1005(lisa),1008(sales)
[root@rhel10 ~]# usermod -aG sales lara && id lara
uid=1006(lara) gid=1006(lara) groups=1006(lara),1008(sales)
[root@rhel10 ~]# usermod -aG sales sara && id sara
uid=1007(sara) gid=1007(sara) groups=1007(sara),1008(sales)
```

7)

`usermod -aG <GROUP\_NAME> <USER\_NAME>` helps us here.

8) `[root@rhel10 /home #] mkdir myfolder`

```
[root@rhel10 /home #] chgrp sales myfolder
[root@rhel10 /home #] chmod g+rw myfolder/
[root@rhel10 /home #] ll
total 4
drwx-----. 3 lara      lara      78 Mar  8 22:08 lara
drwx-----. 3 lisa      lisa      78 Mar  8 22:08 lisa
drwx-----. 14 mr-ravan mr-ravan 4096 Mar  8 04:27 mr-ravan
drwxrwxr-x.  2 root      sales      6 Mar  8 22:21 myfolder ←
drwx-----. 4 networker networker 113 Mar  8 06:46 networker
drwx-----. 3 sara      sara      78 Mar  8 22:09 sara
9) drwx-----. 4 scriptrunner scriptrunner 92 Mar  8 18:17 scriptrunner
```

10) `[root@rhel10 /home #] su - lisa`

```
[lisa@rhel10 ~ $] cd ../myfolder/
[lisa@rhel10 /home/myfolder $] touch lisa-file.txt
[lisa@rhel10 /home/myfolder $] ll
total 0
-rw-r--r--. 1 lisa lisa 0 Mar  8 22:25 lisa-file.txt
11)
```

12) As you can see, the group owner is lisa herself.

13)

```
[root@rhel10 /home #] chmod g+s myfolder/
[root@rhel10 /home #] ll
total 4
drwx-----. 3 lara      lara      78 Mar  8 22:08 lara
drwx-----. 4 lisa      lisa      92 Mar  8 22:25 lisa
drwx-----. 14 mr-ravan mr-ravan 4096 Mar  8 04:27 mr-ravan
drwxrwsr-x.  2 root      sales     27 Mar  8 22:25 myfolder ←
```

Using `chmod g+s <DIRECTORY_NAME>`, we can set a group collaboration.

14) [root@rhel10 /home #] su - lara

```
[lara@rhel10 ~ $] cd ../myfolder/
[lara@rhel10 /home/myfolder $] touch lara-file.txt
[lara@rhel10 /home/myfolder $] ll
total 0
-rw-r--r--. 1 lara sales 0 Mar  9 01:28 lara-file.txt
```

16) As you can see (in task 15, above), the group owner of the file is `sales`. It happens because we set group collaboration for its parent folder.

```
[lara@rhel10 /home/myfolder $] su -
Password:
Last login: Mon Mar  9 01:24:37 +04 2026 on pts/1
17) [root@rhel10 ~ #]
```

18)

```
[root@rhel10 /home/myfolder #] mkdir -p folder1/folder2/folder3
[root@rhel10 /home/myfolder #] tree folder1
folder1
├── folder2
│   └── folder3
3 directories, 0 files
```

19)

```
[root@rhel10 /home/myfolder #] touch folder1/folder2/folder3/myfile
[root@rhel10 /home/myfolder #] ll folder1/folder2/folder3/
total 0
-rw-r--r--. 1 root sales 0 Mar  9 01:38 myfile
```

20) We just need to use `setfacl -m g:wheel:rw <PATH>`. I will show the command in Task 21.

21)

```
[root@rhel10 /home/myfolder/folder1/folder2/folder3 #] getfacl myfile #Before access list
# file: myfile
# owner: root
# group: sales
user::rw-
group::r--
other::r--

[root@rhel10 /home/myfolder/folder1/folder2/folder3 #] setfacl -m g:wheel:rw myfile
[root@rhel10 /home/myfolder/folder1/folder2/folder3 #] getfacl myfile #After access list
# file: myfile
# owner: root
# group: sales
user::rw-
group::r--
group:wheel:rw-
mask::rw-
other::r--
```

22) We need to use Sticky Bit here: `chmod o+t <DIRECTORY_NAME>`.

```
[root@rhel10 /home #] chmod o+t myfolder/
[root@rhel10 /home #] ll
total 4
drwx-----. 4 lara      lara      92 Mar  9 01:27 lara
drwx-----. 4 lisa      lisa      92 Mar  8 22:25 lisa
drwx-----. 14 mr-ravan  mr-ravan 4096 Mar  8 04:27 mr-ravan
drwxrwsr-t.  3 root      sales     63 Mar  9 01:35 myfolder
```

So, when Sara tries to remove Lisa's file:

```
[sara@rhel10 /home/myfolder $] rm -rf lisa-file.txt
rm: cannot remove 'lisa-file.txt': Operation not permitted
```

When we implement Sticky Bit to parent directory, a file can be removed only by its creator or root user.

23)

```
[root@rhel10 /home/myfolder #] chattr +i folder1/folder2/folder3/myfile
[root@rhel10 /home/myfolder #] rm -rf folder1/folder2/folder3/myfile #Even root user can't delete it
rm: cannot remove 'folder1/folder2/folder3/myfile': Operation not permitted
```

`chattr +I <PATH_TO_FILE>`

We use `‘i’` attribute, it makes files immutable. So, nobody can delete them. If we want to delete it, root should remove this attribute.

24)

```
[root@rhel10 / #] # Create a folder and copy contents into it
[root@rhel10 / #] mkdir /yourfolder && cp -r /home/myfolder/. /yourfolder
[root@rhel10 / #] # Backup permissions with getfacl
[root@rhel10 / #] getfacl -R /home/myfolder > /tmp/myfolder_permissions.acl
getfacl: Removing leading '/' from absolute path names
[root@rhel10 / #] # Rewrite paths in the backup file. Otherwise copying fails
[root@rhel10 / #] sed 's|/home/myfolder|yourfolder|g' /tmp/myfolder_permissions.acl > /tmp/yourfolder_permissions.acl
[root@rhel10 / #] # Restore permissions
[root@rhel10 / #] cd / && setfacl --restore=/tmp/yourfolder_permissions.acl
```

As you can see, first we create a folder and copy the contents from `/home/myfolder` to here. Then get the permissions with `getfacl -R /home/myfolder` and redirect it to `/tmp/myfolder_permissions.acl` to save as backup.

We use `sed` command to rewrite paths to prevent confusion in system. At last, we apply changes using `setfacl --restore=/tmp/yourfolder_permissions.acl`.

If we go to `/yourfolder` with `cd` command, run this command:

```
[root@rhel10 /yourfolder #] getfacl -R .
```

We will see that all permissions are successfully applied here.

What does `sed` command do here?

*`sed` here acts as a **path rewriter**. It reads the ACL backup file and replaces every occurrence of `home/myfolder` with `yourfolder` so that when `setfacl --restore` runs, it applies the permissions to `/yourfolder` instead of the original `/home/myfolder`.*

Without `sed`, `setfacl` would try to restore permissions back onto the original `home/myfolder` paths, defeating the whole purpose of the task.

25) Our command to switch between users is `su`. Without any argument it tries to switch to root user. There are two types of `su` command: `su <USERNAME>` or `su - <USERNAME>`.

`su <USERNAME>` → Switches user **but keeps your current environment** (PATH, variables, working directory). We never use it in real life.

`su - <USERNAME>` → Switches user **and loads that user's full login environment** (like a fresh login). It changes HOME, PATH, shell settings, and moves you to that user's home directory. We always use it in real life.

26) 555 → r-x r-x r-x; 400 → r-- --- --- ; 750 → rwx r-x ---

27) Umask shows us default permissions for files and directories when we create them. Its default value is 0022. We calculate permissions for files from 666, for directories it is 777. We basically subtract umask value from these and find the default permissions for files and directories.

Imagine our umask is 011. Then for files default permission is 655, for directories it is 766.

28) The `/etc/skel` directory (skeleton) is a **template directory** for new user home directories. When you create a new user with `useradd`, the contents of `/etc/skel` are **automatically copied** into the new user's home directory. `.bash_profile`, `.bashrc`, `.bash_logout` are example files from this directory.

29) We need to go to `/etc/skel` directory and create a script file here. I created a script file named `mylogout.sh` and add these:

```
[root@rhel10 /etc/skel #] cat mylogout.sh
#!/bin/bash
touch ~/logout_$(date +%Y%m%d_%H%M%S).txt
```

We should make it executable file using: `chmod +x mylogout.sh`

To ensure this file runs in logout, we need to add this line to

`/etc/skel/.bash_logout`:

```
[root@rhel10 /etc/skel #] cat .bash_logout
# ~/.bash_logout
~/mylogout.sh <—————
```

`.bash_logout` file is read when the user logs out. So, by adding this line to `.bash_logout` inside `/etc/skel` make sure our script file will run whenever user logs out.

Checking:

```
[root@rhel10 /etc/skel #] useradd userN && passwd userN
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
[root@rhel10 /etc/skel #] su - userN
[userN@rhel10 ~]$ ll
total 4
-rwxr-xr-x. 1 userN userN 54 Mar  9 03:37 mylogout.sh
[userN@rhel10 ~]$ logout
[root@rhel10 /etc/skel #] su - userN
Last login: Mon Mar  9 03:48:27 +04 2026 on pts/1
[userN@rhel10 ~]$ ll
total 4
-rw-r--r--. 1 userN userN  0 Mar  9 03:48 logout_20260309_034837.txt
-rwxr-xr-x. 1 userN userN 54 Mar  9 03:37 mylogout.sh
[userN@rhel10 ~]$ exit
logout
[root@rhel10 /etc/skel #] su - userN
Last login: Mon Mar  9 03:48:41 +04 2026 on pts/1
[userN@rhel10 ~]$ ll
total 4
-rw-r--r--. 1 userN userN  0 Mar  9 03:48 logout_20260309_034837.txt
-rw-r--r--. 1 userN userN  0 Mar  9 03:48 logout_20260309_034846.txt
-rwxr-xr-x. 1 userN userN 54 Mar  9 03:37 mylogout.sh
```

The script creates a new uniquely named file each time the user logs out (timestamp prevents overwriting).

```
[root@rhel10 ~ #] useradd ravan -e "2026-12-31" && passwd ravan
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
[root@rhel10 ~ #] tail -n1 /etc/shadow #To see the account expiration
ravan:$y$j9T$Ie05cq$HAVT17QAbVZRkR1$1EmXfB2YtVvGJqivCTN0kj4NzsAxpDErIRwwy1EfZi.:20520:0:99999:7::20817:
```

30)

We use `-e` option in `useradd` to set expiration date for an account.

31) To solve this task, we need to go to `/etc/login.defs` file and edit specific lines there. I show edited lines from this file:

```
PASS_MAX_DAYS      90
PASS_MIN_DAYS      2
PASS_MIN_LEN       11
PASS_WARN_AGE      7
```